

Bucket Sort and Radix Sort



Bucket Sort



Time Complexity comparison of Sorting Algorithms

Algorithm	Data Structure	Time Complexity			Space Complexity
		Best	Average	Worst	
Quicksort	Array	O(n log(n))	O(n log(n))	O(n^2)	O(n)/0(logn)
Mergesort	Array	O(n log(n))	O(n log(n))	O(n log(n))	O(n)
Heapsort	Array	O(n log(n))	O(n log(n))	O(n log(n))	O(1)
Bubble Sort	Array	O(n)	O(n^2)	O(n^2)	O(1)
Insertion Sort	Array	O(n)	O(n^2)	O(n^2)	O(1)
Select Sort	Array	O(n^2)	O(n^2)	O(n^2)	O(1)
Bucket Sort	Array	O(n+k)	O(n+k)	O(n^2)	O(nk)
Radix Sort	Array	O(nk)	O(nk)	O(nk)	O(n+k)



Space Complexity comparison of Sorting Algorithms

Algorithm	Data Structure	Worst Case Auxiliary Space Complexity
Quicksort	Array	O(n)
Mergesort	Array	O(n)
Heapsort	Array	O(1)
Bubble Sort	Array	O(1)
Insertion Sort	Array	O(1)
Select Sort	Array	O(1)
Bucket Sort	Array	O(nk)
Radix Sort	Array	O(n+k)

Time complexity of Sorting

- Several sorting algorithms have been discussed, the best so far are:
  - Heap sort and Merge sort: O(n log n)
  - Quick sort (best one in practice): O(n log n) on average,
    : $O(n^2)$ worst case

Can we do better than O(n log n)?

- No.

Any comparison-based sorting algorithm will need at least O(n log n) operations

Restrictions on the problem

- Suppose the values in the list to be sorted can repeat but the values have a limit (e.g., values are digits from 0 to 9)

Sorting in such can be made more efficient

- **Is it possible to come up with an algorithm better than O(n log n)?**
  - Yes , Strategy will not involve comparisons

Bucket sort

Idea:

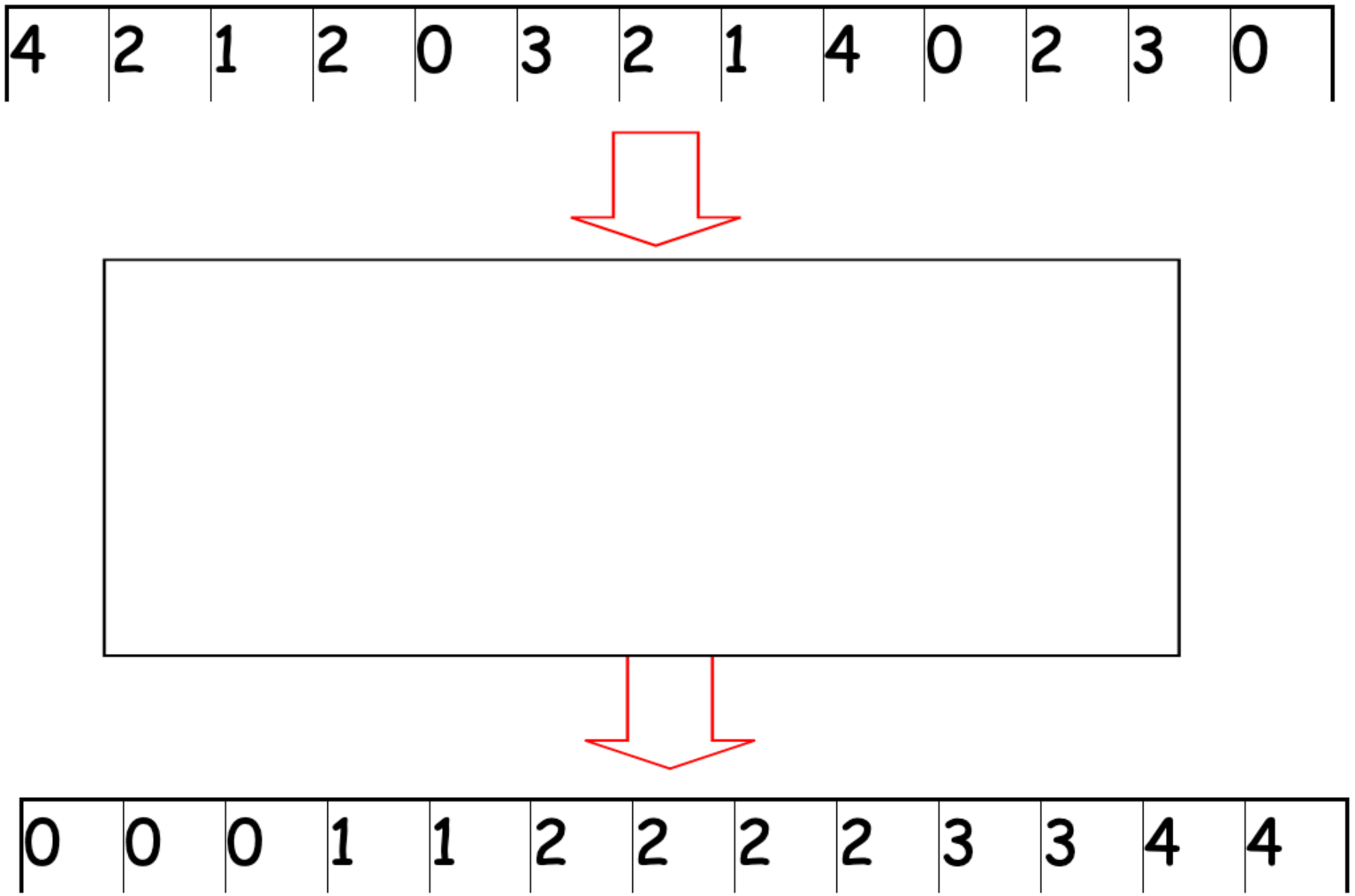
suppose the values are in the range 0..m-1;

- start with m empty *buckets* numbered 0 to m-1,
- scan the list and place element s[i] in bucket s[i], and
- then output the buckets in order

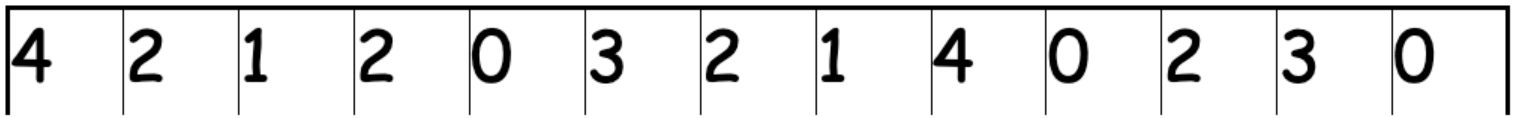
Will need an array of buckets, and the *values in the list* to be sorted *will be the indexes to the buckets*

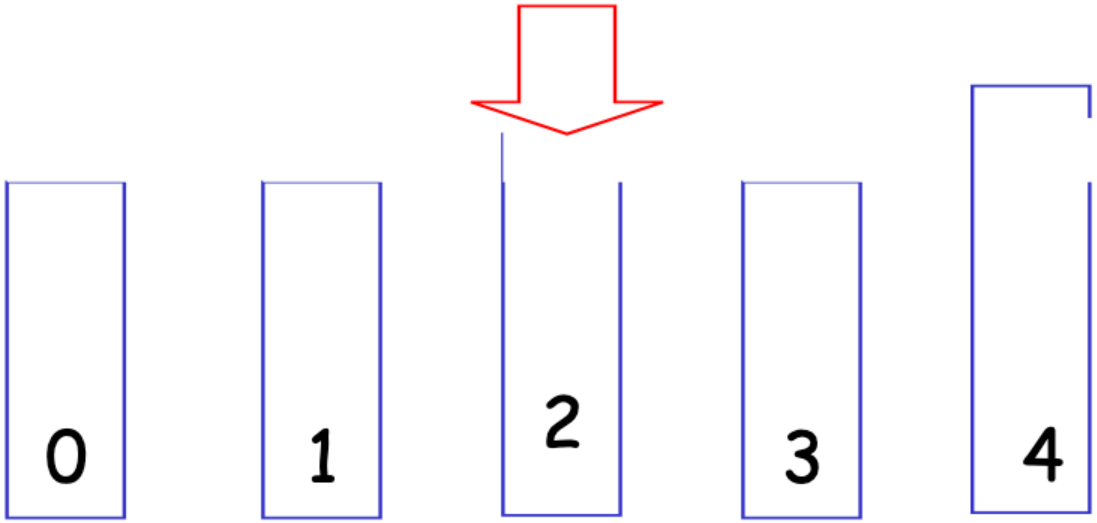
- No comparisons will be necessary

Example



Example

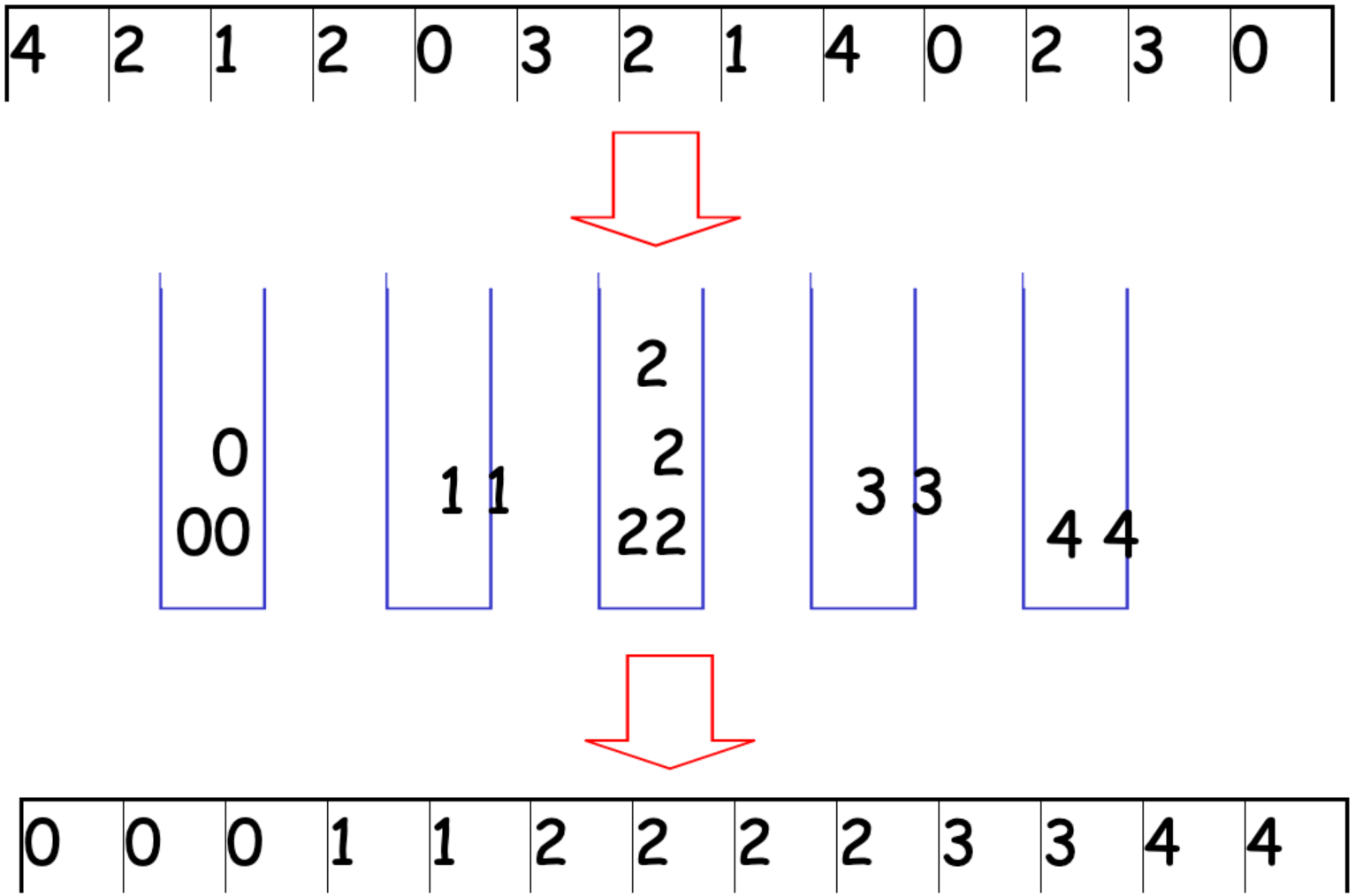




Bucket with indices same as input range



Example



Bucket sort algorithm

Algorithm BucketSort(S)
(values in S are between 0 and m-1)

```
for j ← 0 to m-1 do    // initialize m buckets
        b[j] ← 0

for i ← 0 to n-1 do        // place elements in their
        b[S[i]] ← b[S[i]]  + 1    // appropriate buckets

i ← 0
for j ← 0 to m-1 do        // place elements in buckets
        for r ← 1 to b[j] do    // back in S
                S[i] ← j
                i ← i + 1
```

Values versus entries

- If we were **sorting values**, each *bucket is just a counter* that we increment whenever a value matching the bucket's number is encountered

- If we were **sorting entries** according to keys, then each *bucket is a queue*
  - Entries are enqueued into a matching bucket
  - Entries will be dequeued back into the array after the scan

Time complexity

```
Algorithm BucketSort( S )

( values in S are between 0 and m-1 )


for j ← 0 to m-1 do    // initialize m buckets
            b[j] ← 0
for i ← 0 to n-1 do    // place elements in their
            b[S[i]] ← b[S[i]]  + 1    // appropriate buckets
i ← 0
for j ← 0 to m-1 do    // place elements in buckets
            for r ← 1 to b[j] do    // back in S
                        S[i] ← j
                        i ← i + 10
```

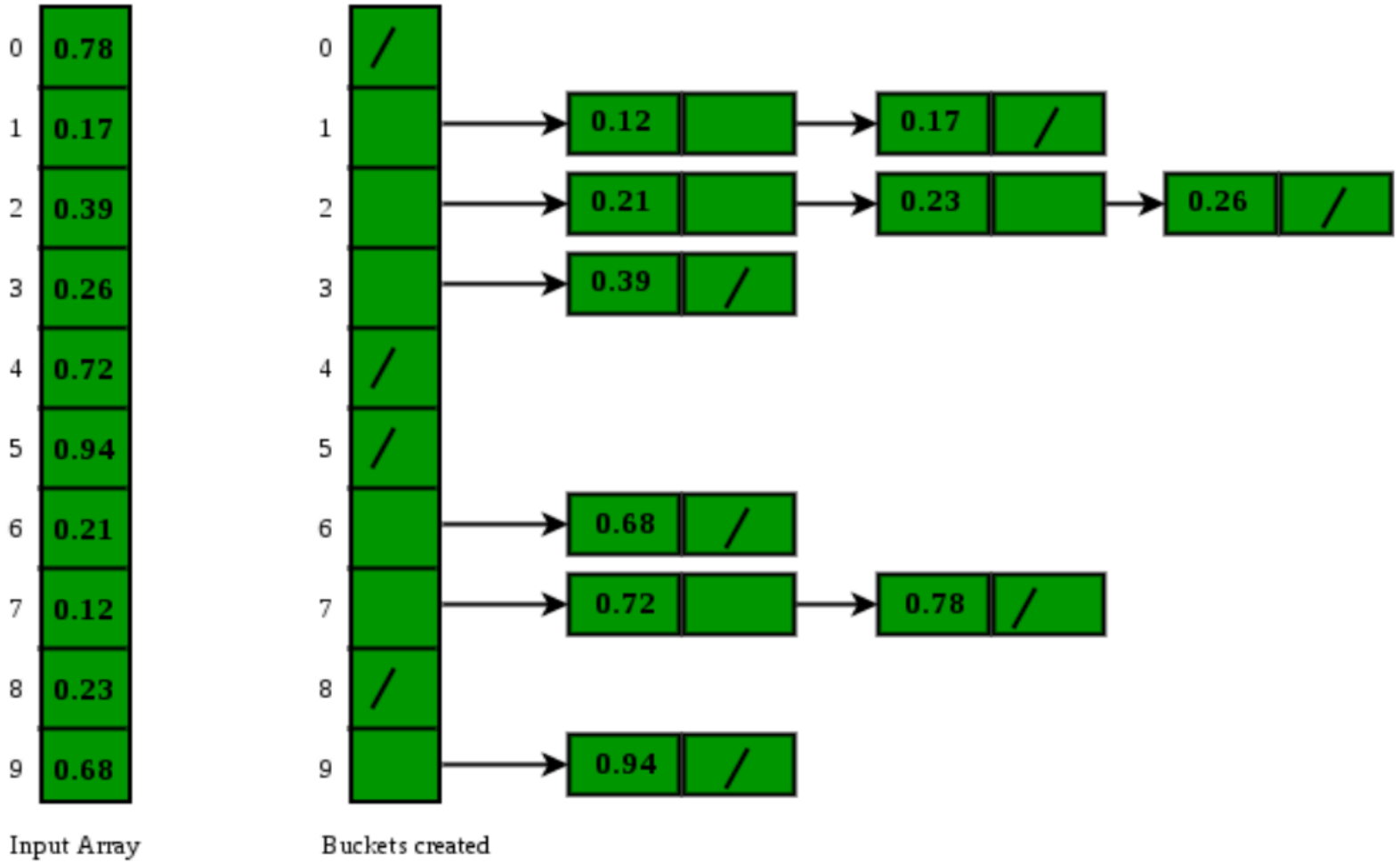
- Bucket initialization : O(m)
- From array to buckets : O(n)
- From buckets to array : O(n)
  - Even though this stage is a nested loop, but all we do is dequeue from each bucket until they are all empty –> n dequeue operations in all

- Since m will likely be small compared to n,

  Bucket sort is O(n)
  - Strictly speaking, time complexity is O (n + m)

Sorting floating point numbers

Sort a large set of floating point numbers which are in range from 0.0 to 0.9 and are uniformly distributed across the range.





Radix sort

Sorting integers

- **To perform bucket sort on any array of (non-negative) integers?**

→number of buckets will depend on the maximum integer value

Example

- sorting ***1000*** integers and the maximum value is ***999999***, you will need *1 million* buckets!

- Time complexity is not really ***O(n)***
- because **m >>>n,** actual time complexity is **O(m)**

Can we do better?

Radix sort

Idea:

- repeatedly sort by digit—
- perform multiple bucket sorts on S starting with the rightmost digit

- If maximum value is 999999, only ten buckets will be necessary (not 1 million)

- Use this strategy when the keys are integers, and there is a reasonable limit on their values
  - **Number of passes** (bucket sort stages) will *depend on the number of digits in the maximum value*

Algorithm for Radix Sort:

1. Determine the maximum number of digits or characters in the elements to be sorted. You can do this by finding the maximum element and counting its digits or characters.

2. Initialize ten buckets (0 to 9) for decimal digits or 256 buckets (0 to 255) for characters in a byte, one for each possible digit or character value.

3. Start from the least significant digit or character position and perform the following steps for each digit or character position, moving from right to left:

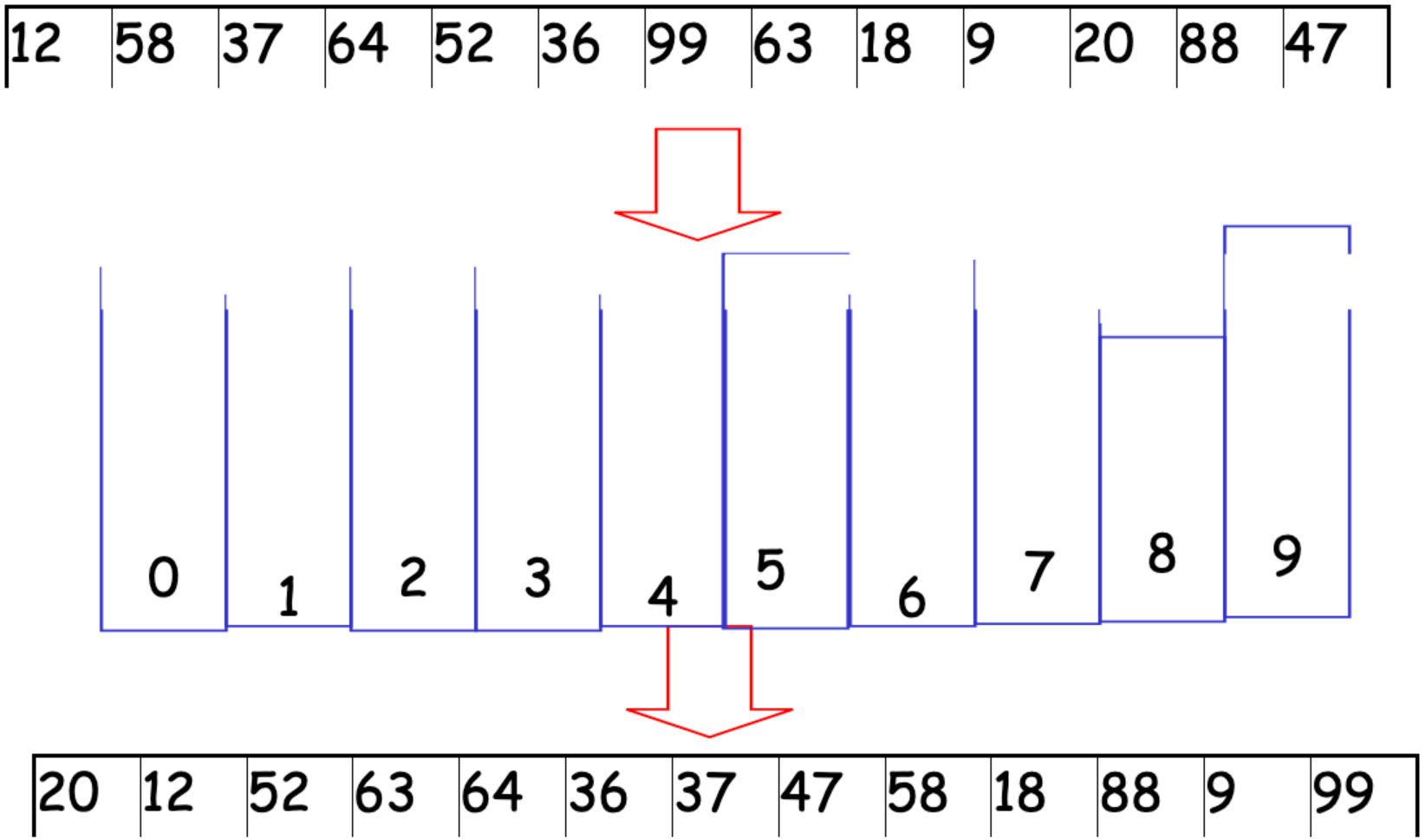
   a. Place each element in the appropriate bucket based on the value of the current digit or character position. For example, if sorting integers, place an integer in the bucket corresponding to the value of its current digit.

   b. Collect the elements from the buckets in the order they were placed. This step preserves the order of elements with the same digit or character values.

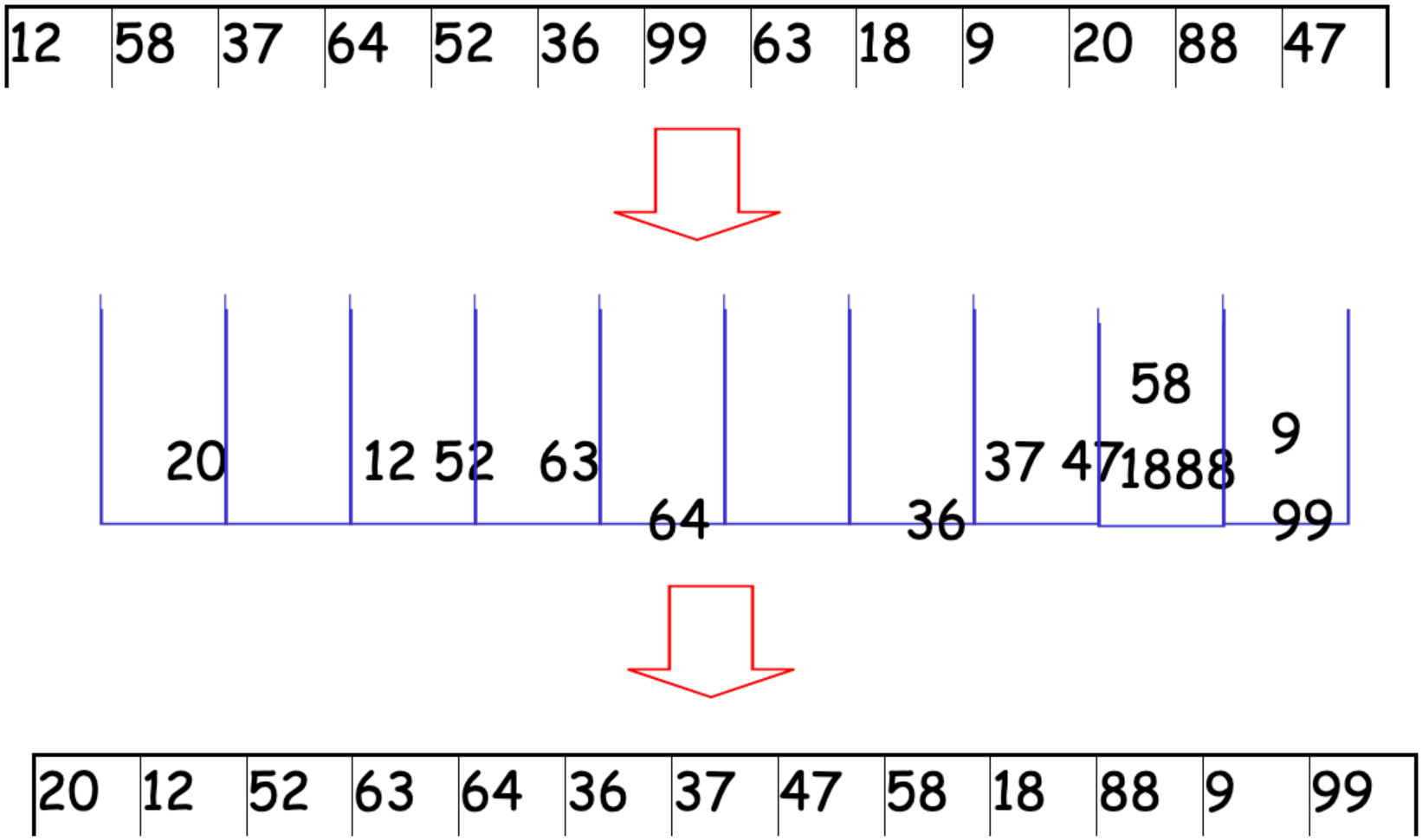
4. Repeat step 3 for each digit or character position, moving from the least significant to the most significant.

5. After processing all digit or character positions, the array will be sorted.

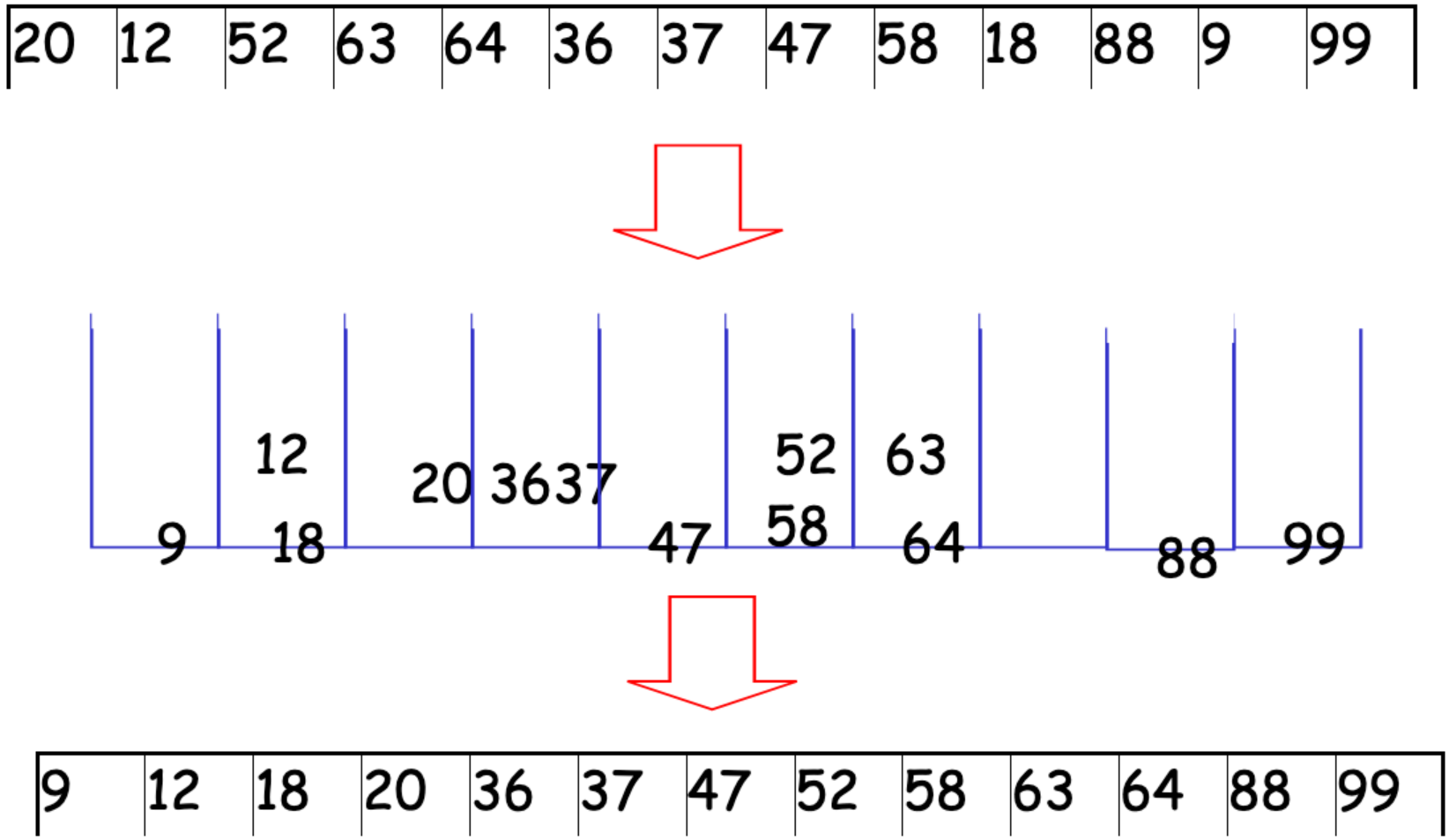
Example: first pass



Example: first pass

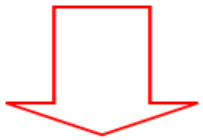


Example: second pass



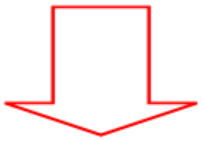
Example: 1st and 2nd passes

12	58	37	64	52	36	99	63	18	9	20	88	47



sort by rightmost digit
(least significant)

20	12	52	63	64	36	37	47	58	18	88	9	99



sort by leftmost digit
(second least significant)

9	12	18	20	36	37	47	52	58	63	64	88	99



Radix sort and stability

- Radix sort works as long as the bucket sort stages are **stable** sorts

- Stable sort: in case of ties, relative order of elements are preserved in the resulting array
  - Suppose there are two elements whose first digit is the same; for example, 52 & 58
  - If 52 occurs before 58 in the array prior to the sorting stage, 52 should occur before 58 in the resulting array

- This way, the work carried out in the previous bucket sort stages is preserved

Time complexity

- If there is a fixed number p of bucket sort stages (six stages in the case where the maximum value is 999999), then radix sort is O(n)
  - There are p bucket sort stages, each taking O(n) time
- Strictly speaking, time complexity is O(pn), where p is the number of digits

Radix sort -summary

- Only 10 buckets are needed regardless of number of stages since the buckets are reused at each stage

- Radix sort can apply to words
  - Set a limit to the number of letters in a word
  - Use 27 buckets (or more, depending on the letters/characters allowed), one for each letter plus a “blank” character
  - The word-length limit is exactly the number of bucket sort stages needed

  Strategy ?
  Read char left to right ? [R_L]
  Blank spaces to be taken at beginning or last ? [L]
  Space bucket to be kept at beginning or last ? [L]



Summary

- Bucket sort and Radix sort are O(n) algorithms only because we have imposed restrictions on the input list to be sorted

- Sorting, in general, can be done in O(nlogn) time



Sorting Words

Using

Radix Sort



Apply *Radix sort* on the following four words

dart, fear, do, car



Radix sort on the following four words

making the following assumptions

- *Read characters in word from right to left*
- *Blank spaces in a word (if assumed) to be taken at last*
- *Space bucket to be kept at last*





Bucket Sort Strategy and Analysis

https://youtu.be/N7-q_O3lgtg

Radix Sort and its Complexity Analysis

https://youtu.be/pdY5CC_3iEc